ASSIGNMENT

Name :- Shweta Mahar

Subject: Java Lab

Course : Bse (Cs) 4th Sem

Roll Number : 24070166

Enrollment Number : GGV / 24 / 05166

Guided By : Dr. Ratnesh Srivastava

Question 1 :

Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input Also perform search operation based on farmers' name.

Code:

```
import java.io.*;
import java.util.*;

class Farmer {
    String name, fruit, city;
    int quantity;

    Farmer(String n, String f, String c, int q) {
        name = n; fruit = f; city = c; quantity = q;
    }
}

public class FarmerData {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        ArrayList<Farmer> list = new ArrayList<>();

        for (int i = 0; i < 3; i++) {
            list.add(new Farmer(sc.next(), sc.next(), sc.next(), sc.nextInt()));
        }

        BufferedWriter bw = new BufferedWriter(new FileWriter("before.txt"));
        for (Farmer f : list)
            bw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" + f.quantity + "\n");
        bw.close();

        list.sort(Comparator.comparingInt(f -> f.quantity));

        bw = new BufferedWriter(new FileWriter("after.txt"));
        for (Farmer f : list)
            bw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" + f.quantity + "\n");
        bw.close();

        String key = sc.next();
```

```
        for(Farmer f : list)
            if(f.name.equalsIgnoreCase(key))
                System.out.println("Found: " + f.name + " " + f.fruit);
    }
}
```

Output:

```
Input:
RamMango Raipur 500
ShyamApple Delhi 200
MohanBanana Pune 800
Ram

Output:
Found: RamMango
before.txt

Ram     Mango    Raipur 500
ShyamApple      Delhi     200
MohanBananaPune          800
after.txt

ShyamApple      Delhi     200
Ram     Mango    Raipur 500
MohanBananaPune          800
```

Question 2 :

Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality

Code:

```
import java.util.*;
        class Fruit {
        Stringname,quality;
        int quantity;
      Fruit(Stringn,int q, String qu) {
        name = n;
        quantity=(q> 1000) ? q / 2 : q;
        quality = qu;
    }
```

```
}

public class FruitSearch {
    publicstaticvoidmain(String[] args) {
        Scannersc=newScanner(System.in);
        ArrayList<Fruit>list=new ArrayList<>();

        list.add(newFruit("Mango", sc.nextInt(), sc.next()));
        list.add(newFruit("Apple", sc.nextInt(), sc.next()));
        list.add(newFruit("Papaya", sc.nextInt(), sc.next()));

        String f = sc.next();
        String q = sc.next();

        for (Fruit fr : list)
            if(fr.name.equalsIgnoreCase(f) && fr.quality.equalsIgnoreCase(q))
                System.out.println("Found with quantity: " + fr.quantity);
    }
}
```

Output:

```
Input:
1200 good
500 average
800 bad
Mango good

Output:
Found with quantity: 600
```

Question 3 :

There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively. The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

Code:

```
import java.util.*;
  class Node {
      Stringorchard,fruit,quality;
      Node(Stringo,Stringf,String q) {
     orchard=o;fruit=f;quality = q;

    }
  }
```

```
publicclassOrchardTree {

 publicstatic void main(String[] args) {

   ArrayList<Node> list = new ArrayList<>();
   list.add(new Node("A", "Apple", "good"));
   list.add(new Node("B", "Litchi", "good"));
   list.add(new Node("C", "Mango", "good"));
   list.add(new Node("D", "Apple", "bad"));

   Scanner sc = new Scanner(System.in);
   Stringkey = sc.next();

        for(Node n : list)

             if(n.fruit.equalsIgnoreCase(key))

              System.out.println(n.orchard + " -> " + n.fruit);

        System.out.println("Tree:");
        for(Node n : list)

           System.out.println("|-- " + n.orchard + "(" + n.fruit + ")");
      }
   }
```

Output:

```
Input:
Apple

Output:
A -> Apple
D -> Apple

Tree:
|-- A(Apple) |-
-  B(Litchi)  |--
C(Mango)   |--
D(Apple)
```

Question 4 :

Take a map provided as a graph & display all the subtrees of connected points.

Code:

```
import java.util.*;
```

```
publicclass GraphSubtree {
    staticvoid dfs(int v, boolean[] visited, List<Integer>[] g) {
        visited[v] = true;
        System.out.print(v + " ");
        for(int n : g[v])
            if(!visited[n]) dfs(n, visited, g);
    }

    publicstatic void main(String[] args) {
        intV= 5;
        List<Integer>[] g = new ArrayList[V];
        for(int i = 0; i < V; i++) g[i] = new ArrayList<>();

        g[0].add(1);
        g[1].add(2);
        g[3].add(4);

        boolean[] visited = new boolean[V];

        for(int i = 0; i < V; i++) {
            if(!visited[i]) {
                System.out.print("Subtree: ");
                dfs(i, visited, g);
                System.out.println();
            }
        }
    }
}
```

Output:

```
Subtree: 0 1 2
Subtree: 3 4
```

Question 5 :

Enter entry point & goal point & produce the matching subtree of the graph.

Code:

```
import java.util.*;
```

```
public class PathDFS {
    staticbooleandfs(intv,intgoal,boolean[] visited, List<Integer>[] g) {
        visited[v]        =        true;
        System.out.print(v + " ");
        if (v == goal) return true;

        for (int n : g[v])
            if(!visited[n]&&dfs(n,goal,visited, g))
                return true;

        return false;
    }

    public static void main(String[] args) {
        int V = 5;
        List<Integer>[] g = new ArrayList[V];
        for(inti=0;i<V;i++)g[i]=newArrayList<>();

        g[0].add(1);
        g[1].add(2);
        g[2].add(3);

        Scannersc=newScanner(System.in);   int
        start = sc.nextInt();
        int goal = sc.nextInt();

        dfs(start, goal, new boolean[V], g);
    }
}
```

Output:

```
Input:
0 3

Output:
0 1 2 3
```

Question 6 :

Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

Code:

```
import java.util.*;

class Pair {
    int node, dist;
    Pair(intn,intd){node = n; dist = d; }
}

publicclassDijkstraAlgo {
    publicstaticvoidmain(String[] args) {
        int V = 5;
        List<Pair>[]g=new ArrayList[V];
        for(inti=0;i<V; i++) g[i] = new ArrayList<>();

        g[0].add(newPair(1,    4));
        g[0].add(newPair(2,    1));
        g[2].add(newPair(1, 2));

        int[]dist=newint[V];
        Arrays.fill(dist,Integer.MAX_VALUE);
        dist[0] = 0;

        PriorityQueue<Pair> pq = new PriorityQueue<>(Comparator.comparingInt(p -> p.dist));
        pq.add(newPair(0, 0));

        while(!pq.isEmpty()) {
        Pairp=pq.poll();
        for(Pairnei:g[p.node]) {
                if(dist[nei.node] > p.dist + nei.dist) {
                    dist[nei.node] = p.dist + nei.dist;
                    pq.add(new Pair(nei.node, dist[nei.node]));
                }
            }
        }

        System.out.println("Shortest distance to 1: " + dist[1]);
    }
}
```

Output:

```
Shortestdistanceto1: 3
```

Question 7:

Write a program to demonstrate the use of all keywords made for exception handling .

Code:

```
publicclassExceptionDemo {
    publicstaticvoidmain(String[] args) {
        try{
            inta=10/0;
        }catch(ArithmeticException e) {
            System.out.println("Caught exception");
        } finally {
            System.out.println("Finally executed");
        }

        thrownewRuntimeException("Manual Exception");
    }
}
```

Output:

```
Caught exception
Finally executed
Exceptioninthread"main" java.lang.RuntimeException: Manual Exception
```

Question 8 :

Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

Code:

```
classNoMoneyException extends Exception {
    NoMoneyException(String msg) { super(msg); }
}

publicclass Bank {
    publicstatic void withdraw(int balance, int amount) throws NoMoneyException {
        if(amount > balance)
            throw new NoMoneyException("Insufficient balance");
        else
            System.out.println("Withdrawn: " + amount);
    }

    publicstatic void main(String[] args) {
```

```
        try{
            withdraw(1000, 1500);
        }catch (NoMoneyException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
Insufficient balance
```

Question 9:

Write a program which enables to use a platform by a train one at a time.

Code:

```
classPlatform {
    synchronized void use(String train) {
        System.out.println(train + " using platform");
        try{Thread.sleep(1000); } catch(Exception e){}
        System.out.println(train + " leaving");
    }
}

publicclass Train {
    publicstatic void main(String[] args) {
        Platform p = new Platform();

        Runnable r = () -> p.use(Thread.currentThread().getName());

        newThread(r,    "Train1").start();
        newThread(r, "Train2").start();
    }
}
```

Output:

```
Train1using platform
Train1leaving
Train2using platform
Train2leaving
```

Question 10 :

Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free

Code:

```
 class Restaurant {
 boolean free = true;
synchronizedvoidorder(String name) throws Exception {
if (!free) wait();
free = false;
System.out.println(name + " ordering");
Thread.sleep(1000);
free = true;
notify();
}
}

public class Hotel {
publicstaticvoidmain(String[] args) {
Restaurantr=newRestaurant();

Runnablewaiter=()-> {
try{
r.order(Thread.currentThread().getName());
}catch(Exceptione){}
};

newThread(waiter,"Customer1").start();
newThread(waiter,"Customer2").start();
}
}
```

Output:

```
Customer1 ordering
Customer2                    ordering
(ordermayvaryduetomultithreading)
```